Karmaveer Punjababa Goverdhane
Arts, Commerce and Science College

**DEPARTMENT OF COMPUTER SCIENCE**

# Shiv Shakti Port Scanner

A Multi-Threaded Network Diagnostic

& Securite Tool

## *PROJECT REPORT*

**PROJECT GUIDE**

**Prof. Miss. Bhagyashree N. Sonar**

**SUBMITTED BY:**

**Chavhan Prathamesh Chhabu**

**Submission Date:**

**S.Y.B.Sc. (Computer Science) Mini Project**

**Academic Year (2025 - 2026 )**

# Project Title: Shiv Shakti Port Scanner

**Name : Chavhan Prathamesh Chhabu**

**Roll No: 14 Exam Seat No:**

**Project Guide Name: Prof.Miss.Bhagyashree Sonar**

**Project Guide Signature:**

**Start Date:** **Completion Date:**

# DEPARTMENT OF COMPUTER SCIENCE

## CERTIFICATE

This is to certify that Mr. **Chavhan Prathamesh Chhabu** has successfully and satisfactorily completed and submitted the field project titled **Shiv_Shakti_Port_Scanner** in partial fulfilment of **S.Y.B.Sc.(CS) Lab course CS-281-FP Mini Project Semester IV** prescribed by **Savitribai Phule Pune University** during the academic year (2025-2026)

(Project Guide) (H.O.D. Computer Science)

INTERNAL EXAMINER EXTERNAL EXAMINER

# ACKNOWLEDGEMENT

I would like to take this opportunity to express my profound gratitude and sincere respect to my project guide, **Prof.Miss.Bhagyashree.N.Sonar** , for her invaluable mentorship, constant encouragement, and insightful guidance throughout the duration of this project. His/her expertise was instrumental in navigating the complexities of the project and bringing it to a successful conclusion.

I am also deeply grateful to our Head of Department, **Prof.C. D.Chaudhari**, for providing the necessary facilities and fostering a conducive environment for learning and research within the department.

I would also like to extend my thanks to all the faculty members of the Computer Science department for their invaluable support and for imparting the knowledge that formed the foundation of this work.

Finally, I wish to express my heartfelt thanks to my family and friends for their unwavering support, patience, and motivation, which has been a constant source of strength for me.

**NAME: Chavhan Prathamesh Chhabu**

**ROLL NO.: 14**

# Chapter 1: Abstract

**1. Core Networking Theory**

At its heart, the tool relies on the **TCP Three-Way Handshake**. While sophisticated scanners like Nmap use "Stealth" (SYN) scans, the Shiva-Shakti scanner utilizes the **TCP Connect Scan**.

- **The Mechanism:** The scanner attempts to complete the full handshake using Python's socket.connect() method.
- **Port States:**
    - **Open:** If the target sends a SYN-ACK, the scanner sends an ACK, confirming a service is listening.
    - **Closed:** If the target sends an RST (Reset) packet, the port is active but no service is hosted there.
    - **Filtered:** If no response is received (timeout), a firewall is likely dropping the packets.

---

**2. Architecture: Multi-threading and Queues**

The primary challenge of a port scanner is latency. Scanning 1,000 ports sequentially might take minutes if each timeout lasts one second. The Shiva-Shakti scanner overcomes this using a **Producer-Consumer model**.

**The Multi-threaded Workflow:**

1. **Queue Initialization:** A queue.Queue() object is populated with the range of port numbers (e.g., 1–1024).
2. **Worker Threads:** Multiple threads are spawned simultaneously. Each thread "consumes" a port from the queue.
3. **Atomic Operations:** Because the queue module in Python is thread-safe, multiple threads can pull data without interfering with one another, preventing "race conditions."
4. **Efficiency Gain:** By using 100 threads, the total scan time is theoretically reduced by a factor of 100.

---

**3. The GUI Framework: Tkinter Integration**

To make the tool accessible, the **Tkinter** GUI abstracts the command-line complexity.

- **Input Validation:** The interface ensures the user enters a valid IPv4 address or domain name (resolving it via socket.gethostbyname).

- **Real-time Feedback:** Using a Text widget or Treeview, the scanner prints results as they are found, rather than making the user wait for the entire process to finish.
- **Threading Hazards:** A critical development hurdle in Tkinter is that the GUI freezes if the scan runs on the main thread. The Shiva-Shakti scanner solves this by running the scanning logic on a **background thread**, keeping the "Start/Stop" buttons responsive.

---

**4. Implementation Details**

The technical stack is chosen for its portability and readability:

| **Component** | **Technology** | **Purpose** |
|---|---|---|
| **Language** | Python 3.x | Rapid development and extensive library support. |
| **Networking** | socket module | Interface for TCP/IP communication. |
| **Concurrency** | threading | Parallel execution of port probes. |
| **Data Handling** | queue | Managing the list of ports to be scanned. |
| **Interface** | tkinter | Providing a user-friendly dashboard. |

---

**5. Security & Ethical Implications**

The Shiva-Shakti Port Scanner is designed as a **Network Audit Tool**.

- **Attack Surface Mapping:** By identifying open ports like 21 (FTP), 22 (SSH), or 80 (HTTP), administrators can close unnecessary entry points.
- **Service Mapping:** The tool cross-references open ports against the common_ports dictionary to identify what software is likely running, aiding in vulnerability assessment.

- **Ethical Boundary:** Users are reminded that scanning networks without explicit authorization is illegal and unethical. This tool is intended for educational labs and authorized infrastructure testing.

---

**6. Future Enhancements**

To evolve the Shiva-Shakti scanner, the following features could be integrated:

- **Banner Grabbing:** Fetching the specific version of the service (e.g., Apache 2.4.41) to check against CVE databases.
- **UDP Scanning:** Probing stateless ports, though this is significantly slower and less reliable than TCP.
- **Export Functionality:** Saving scan logs into CSV or JSON formats for professional reporting.

How detailed would you like the technical implementation section to be—should we look at the specific Python code for the thread-worker function?

---

# Chapter 2: Introduction

### 2.1 Background: The Logical Gateway

In the OSI model, ports live at the **Transport Layer (Layer 4)**. Think of an IP address as the street address of an apartment building, while the **Port Number** is the specific apartment number.

- **The Scale of the Problem:** With **65,535** possible TCP and UDP ports, manually checking for vulnerabilities is impossible.
- **The Taxonomy of Ports:**
    - **Well-Known Ports (0–1023):** Reserved for core services like SSH (22), HTTPS (443), and DNS (53).
    - **Registered Ports (1024–49151):** Used by specific applications like MySQL (3306) or MS SQL (1433).
    - **Dynamic/Private Ports (49152–65535):** Temporary ports assigned for client-side communication.

---

### 2.2 Problem Statement: The "Sequential" Bottleneck

The primary hurdle for entry-level network auditing tools is the **Blocking I/O** nature of standard network requests.

#### The Inefficiency of Serial Processing

In a standard sequential script, the program flow looks like this:

1. **Request** port 80 → **Wait** for timeout (2 seconds).
2. **Request** port 81 → **Wait** for timeout (2 seconds). To scan just 100 closed ports, a user would wait over 3 minutes. This latency makes simple scripts useless for large-scale enterprise audits.

#### The "CLI Barrier"

While professional tools like **Nmap** or **Masscan** are incredibly powerful, they require a steep learning curve involving complex flags (e.g., nmap -sS -Pn -T4). For a student or a junior sysadmin, this complexity often leads to "tool fatigue" or incorrect command usage that yields false negatives.

---

### 2.3 Objectives: Designing the Solution

#### 1. Robust Connection Engine

The engine must utilize the socket.socket(socket.AF_INET, socket.SOCK_STREAM) constructor. This ensures the scanner uses the **IPv4** family and the **TCP** protocol. It must be tuned with a strict socket.settimeout() to ensure a single unresponsive port doesn't hang the entire multi-threaded process.

**2. The Queue-Worker Architecture**

To achieve concurrency without overwhelming the system's CPU, the project implements a **Thread Pool** via a Queue.

- **The Queue:** Acts as a synchronized "To-Do List" for all 65,535 ports.
- **The Workers:** A set of 100–200 threads that continuously pull from the queue. When Thread A finishes checking Port 80, it immediately grabs the next available port from the queue, ensuring zero idle time.

**3. Human-Centric UI (Tkinter)**

The GUI is designed to provide immediate visual gratification.

- **Color-Coded Status:** Open ports appear in green, while errors or closed ports appear in red or are filtered out.
- **Progress Tracking:** Unlike a terminal that might stay blank for minutes, the GUI includes a progress indicator or a live-updating text box.

**4. Resilient Input Sanitization**

A major objective is "fail-safe" operation. The tool must use regular expressions (Regex) or the ipaddress library to:

- Validate that an input is a proper IP (e.g., 192.168.1.1).
- Handle DNS resolution, allowing the user to type google.com and automatically converting it to an IP address before the scan begins.

---

**2.4 Technical Constraints & Scope**

To keep the project focused, the following boundaries were established:

- **Protocol:** Focus is strictly on **TCP Connect** scans for reliability.
- **Environment:** Cross-platform compatibility (Windows, Linux, macOS) by sticking to Python’s standard library.
- **Safety:** The tool includes a "Thread Limiter" to prevent the user from accidentally launching a Denial of Service (DoS) attack against their own gateway by opening too many simultaneous connections.

---

# Chapter 3: Networking Fundamentals

# (The Science of Scanning)

### 3.1 The OSI Model: Where the Scanner Lives

The OSI model is a conceptual framework that standardizes the functions of a telecommunication system. The Shiva-Shakti tool primarily interacts with two specific layers to navigate the network.

**Layer 3: The Network Layer (The Address)**

This layer handles the routing of data. The scanner uses the **Internet Protocol (IP)** to locate the specific host on a local network or across the internet. Without Layer 3, the scanner would have a "message" but no "destination."

**Layer 4: The Transport Layer (The Delivery)**

This is the "engine room" of the scanner. The **Transmission Control Protocol (TCP)** ensures that data is delivered reliably.

- **Port Logic:** Ports are logical abstractions at this layer that allow a single IP address to host multiple distinct services (like a web server and an email server) simultaneously.

---

### 3.2 The TCP Three-Way Handshake: The Logic of Discovery

The **Shiva-Shakti Scanner** performs what is known as a **Full Connect Scan**. It completes the entire handshake process to confirm a port's status beyond a doubt.

**The Handshake Sequence:**

1. **SYN (Synchronize):** The scanner sends a packet with the SYN flag set to a specific port. This is essentially asking, *"Are you there? I'd like to talk."*
2. **SYN-ACK (Synchronize-Acknowledge):** * **If Open:** The target replies with a SYN-ACK. This confirms the service is ready.
    - **If Closed:** The target replies with an **RST (Reset)** packet, telling the scanner, *"There is no one here, go away."*

3. **ACK (Acknowledge):** To be a polite "neighbor" on the network, the scanner sends an ACK to complete the connection, and then immediately sends a **FIN (Finish)** or **RST** to close it, preventing the target's resources from being tied up.

**Note on "Stealth":** Unlike the "Half-Open" (SYN) scans used by professional tools like Nmap (which never send the final ACK to avoid being logged), the Shiva-Shakti scanner completes the handshake. This makes it more reliable for educational purposes, though it is more likely to be noticed by a system administrator.

---

**3.3 Port Classifications: The "Phonebook" of Services**

To provide meaningful output, the scanner maps numerical port findings to human-readable services using a lookup table (often based on the IANA registry).

| Range | Category | Common Examples |
|---|---|---|
| **0 – 1023** | **Well-Known** | **22 (SSH):** Secure login.<br>**80 (HTTP):** Unsecured web traffic.<br>**443 (HTTPS):** Secured web traffic. |
| **1024 – 49151** | **Registered** | **3306 (MySQL):** Database access.<br>**3389 (RDP):** Remote Desktop.<br>**5900 (VNC):** Screen sharing. |

| Range | Category | Common Examples |
|---|---|---|
| **49152 – 65535** | **Dynamic** | These are usually "ephemeral." When you browse a website, your computer opens one of these high ports to receive the incoming data from the web server. |

---

**3.4 Determining Port Status**

When the Shiva-Shakti scanner probes a port, it categorizes the result into one of three distinct states:

1. **Open:** The target is actively listening and responded with a SYN-ACK.
2. **Closed:** The target is reachable but no service is listening (RST received).
3. **Filtered/Timed Out:** The scanner sent a SYN packet but heard nothing back. This usually indicates a **firewall** is silently dropping the packets to hide the host's existence.

---

# Chapter 4: Requirement Analysis

### 4.1 Functional Requirements: Defining "What" the Tool Does

Functional requirements outline the specific features and behaviors the system must exhibit to satisfy user needs.

**1. Dynamic Target Selection**

The tool must be intelligent enough to bridge the gap between human-readable names and machine-readable addresses.

- **DNS Resolution:** Using Python's socket.gethostbyname(), the system must resolve domains (e.g., scanme.nmap.org) to their respective IPv4 addresses.
- **Localhost Support:** It must support internal loopback addresses (127.0.0.1) for developers testing their own local services.

**2. Granular Range Customization**

Scanning all 65,535 ports is often a waste of bandwidth and time.

- **User Inputs:** The GUI must provide two distinct input fields: Start Port and End Port.
- **Validation:** The system should prevent illogical inputs (e.g., a Start Port of 80 and an End Port of 20) and restrict inputs to the valid range of 1–65535.

**3. Multi-threaded Execution Engine**

This is the "Shakti" (power) of the scanner.

- **The connect_ex Method:** Unlike connect(), which raises an exception on failure, socket.connect_ex() returns an error code (0 for success). This is more efficient for high-speed scanning.

- **Worker Spawning:** The system must be capable of managing a pool of threads (typically 100–200) that operate concurrently without colliding.

**4. Interactive Log Output**

Information is only useful if it is readable.

- **Live Updates:** Results should be appended to a ScrolledText widget in real-time.
- **Service Mapping:** As an open port is found, the tool should automatically append the likely service name (e.g., Port 80: [OPEN] - HTTP).

---

**4.2 Non-Functional Requirements: Defining "How" the Tool Performs**

Non-functional requirements specify the quality attributes and constraints of the system, such as performance and reliability.

**1. Speed and Throughput**

The tool is built to beat the "sequential" bottleneck.

- **Performance Benchmark:** By utilizing a thread pool, the scanner aims to process 100 ports in under 2 seconds.
- **Latency Management:** The speed is achieved by parallelizing the "wait time" associated with closed or filtered ports.

**2. UI Responsiveness (The "No-Freeze" Policy)**

In many basic Python GUI apps, the window becomes "Unresponsive" while a heavy task runs.

- **Asynchronous Design:** The scanning logic must be offloaded to a **Daemon Thread**. This allows the Tkinter main loop to continue listening for user events (like clicking "Stop" or moving the window) while the scan progresses in the background.

**3. Robustness and Timeout Handling**

A port scanner is only as good as its patience.

- **Timeout Tuning:** A default timeout of **0.3 to 0.5 seconds** is implemented. If it's too long, the scan is slow; if it's too short, it may miss open ports on high-latency networks (False Negatives).
- **Exception Shielding:** The scanner must gracefully handle socket.gaierror (address-related errors) and KeyboardInterrupt without crashing the entire GUI.

**4. Resource Efficiency**

While powerful, the scanner should not "starve" the host machine.

- **CPU Utilization:** The tool should manage threads effectively so that it doesn't consume 100% of the CPU, allowing the user to perform other tasks while the audit runs.
- **Thread Safety:** The use of the queue module ensures that memory is handled safely and ports are not double-scanned or skipped.

---

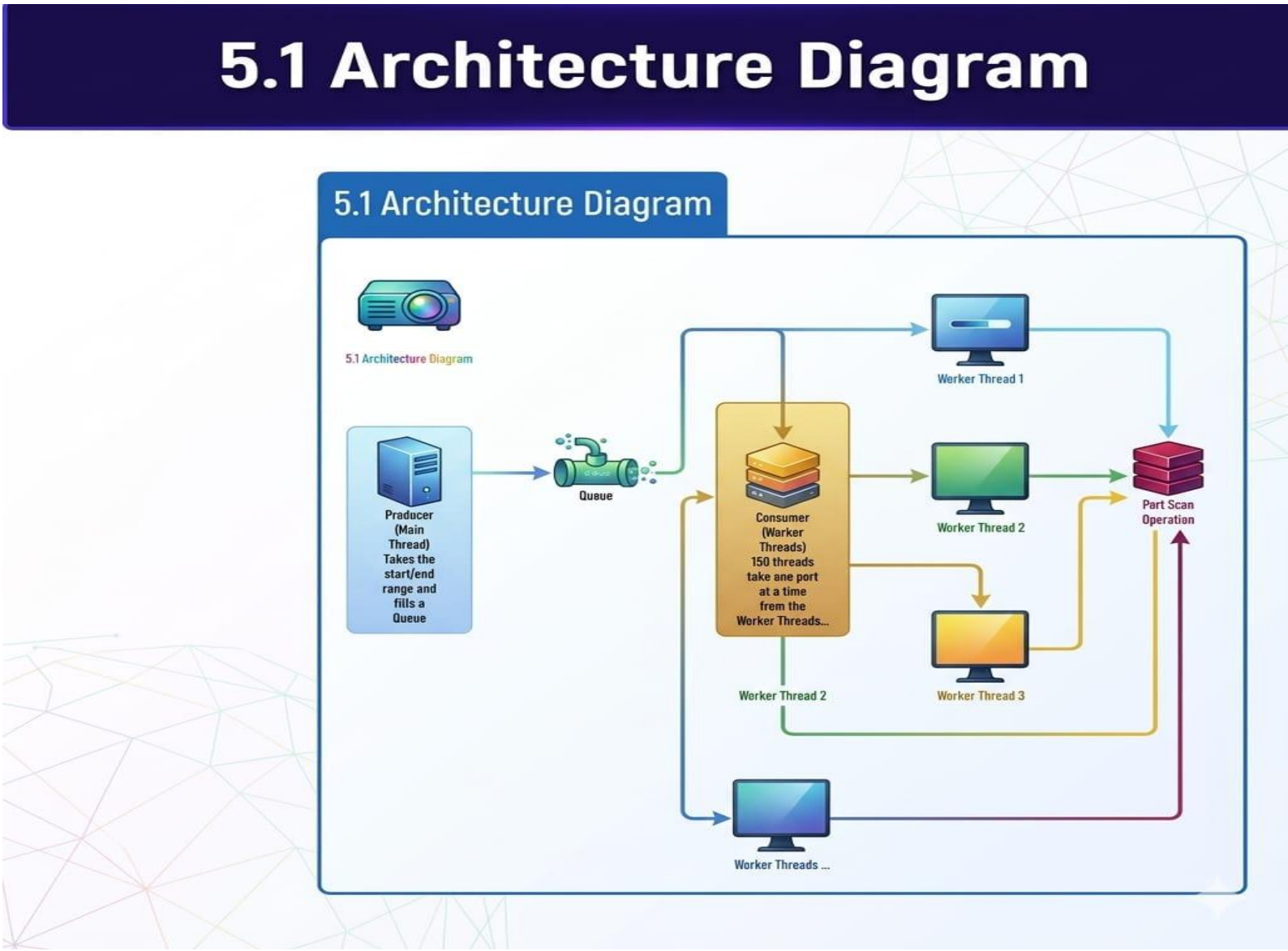
### 4.3 Summary Table of Requirements

| Requirement | Priority | Implementation Method |
|---|---|---|
| **Hostname Resolution** | High | socket.gethostbyname |
| **Concurrency** | High | threading.Thread & queue.Queue |
| **Non-Blocking GUI** | High | Thread separation (Main vs. Background) |
| **Input Validation** | Medium | Regex and try-except blocks |
| **Port Exporting** | Low | Future enhancement (Save to .txt) |

---

# Chapter 5: System Design & UML Diagrams

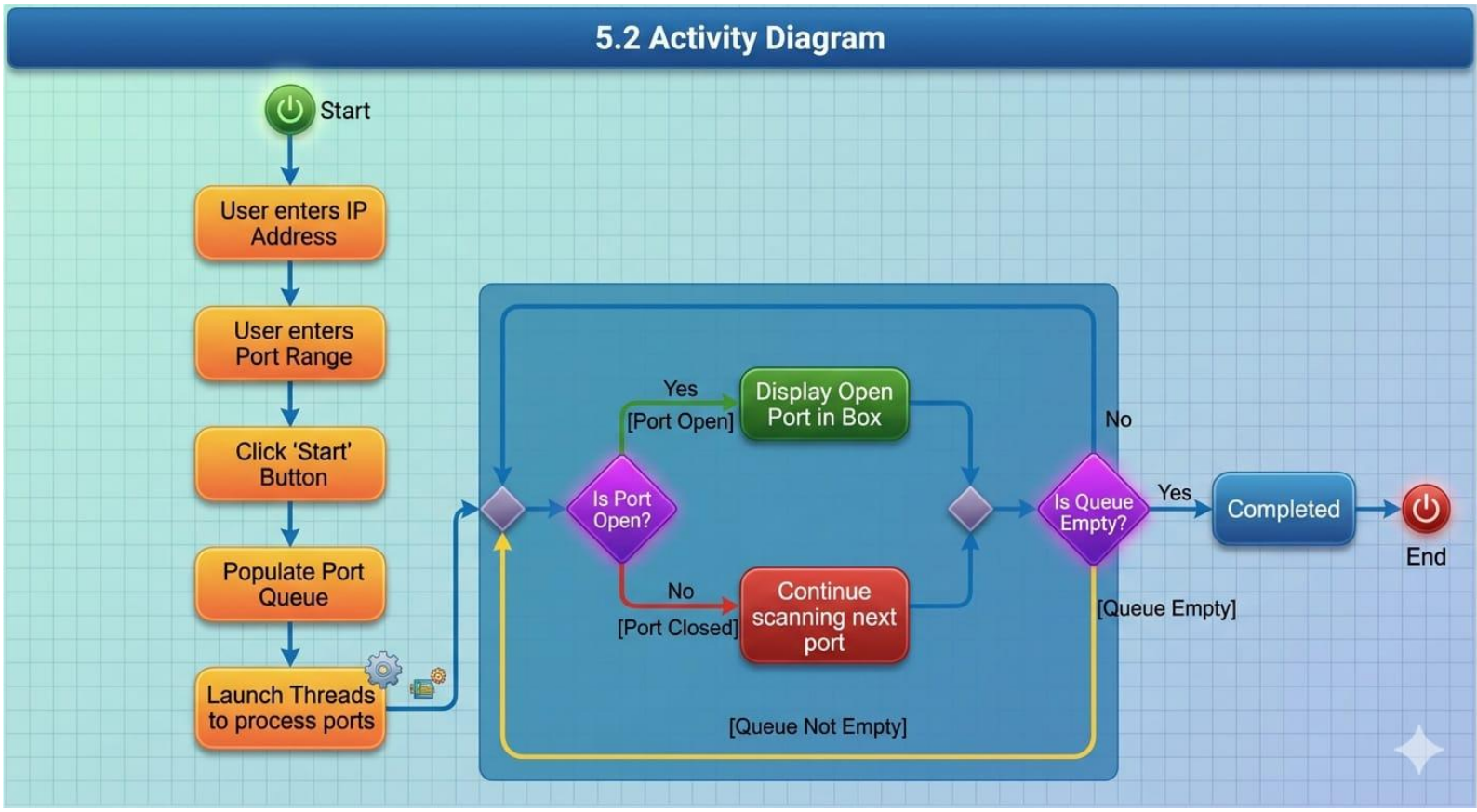
### 5.1 Architecture Diagram



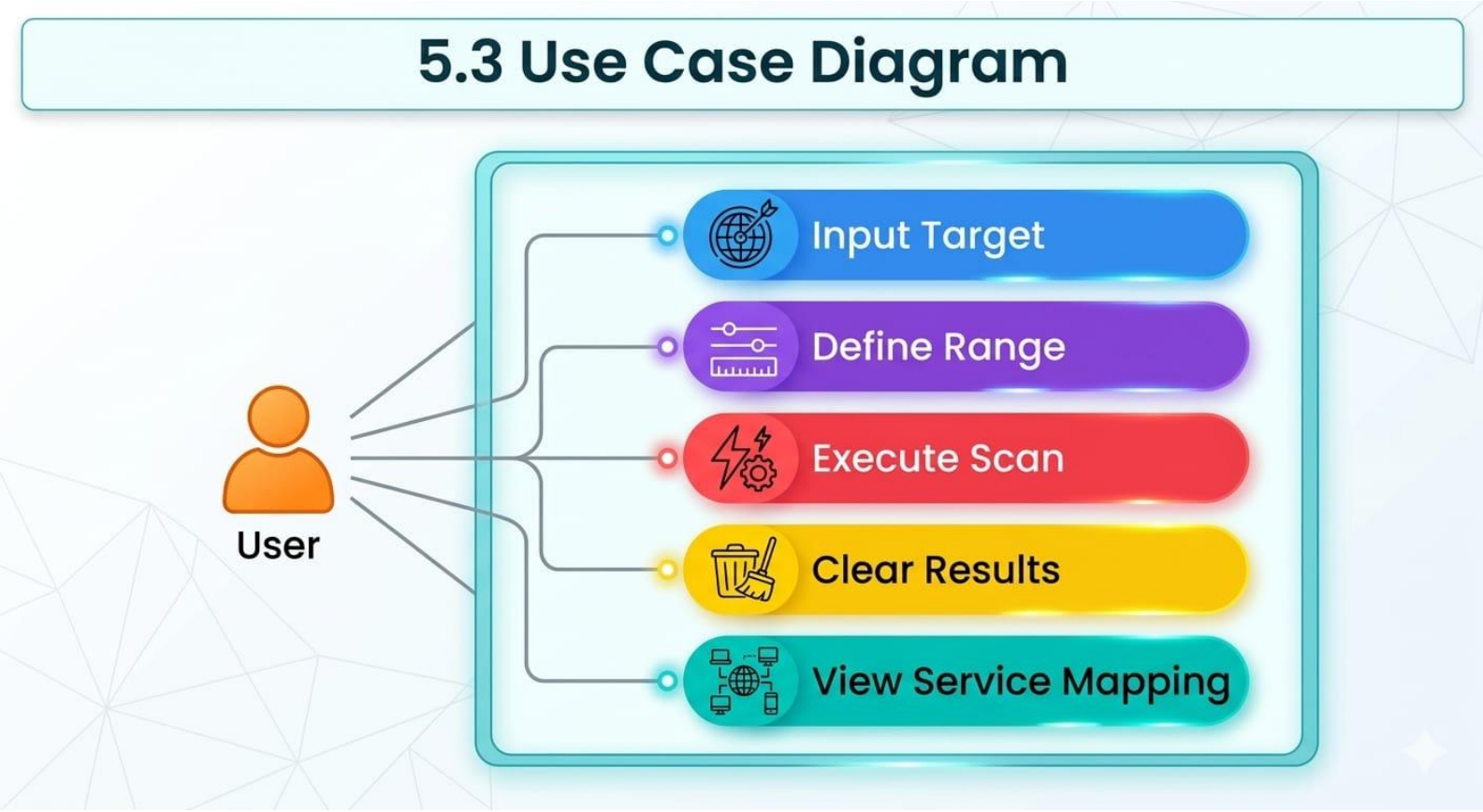

The system follows a **Producer-Consumer Threading Model**.

- **Producer (Main Thread):** Takes the start/end range and fills a Queue.
- **Consumer (Worker Threads):** 150 threads take one port at a time from the queue and perform the scan.

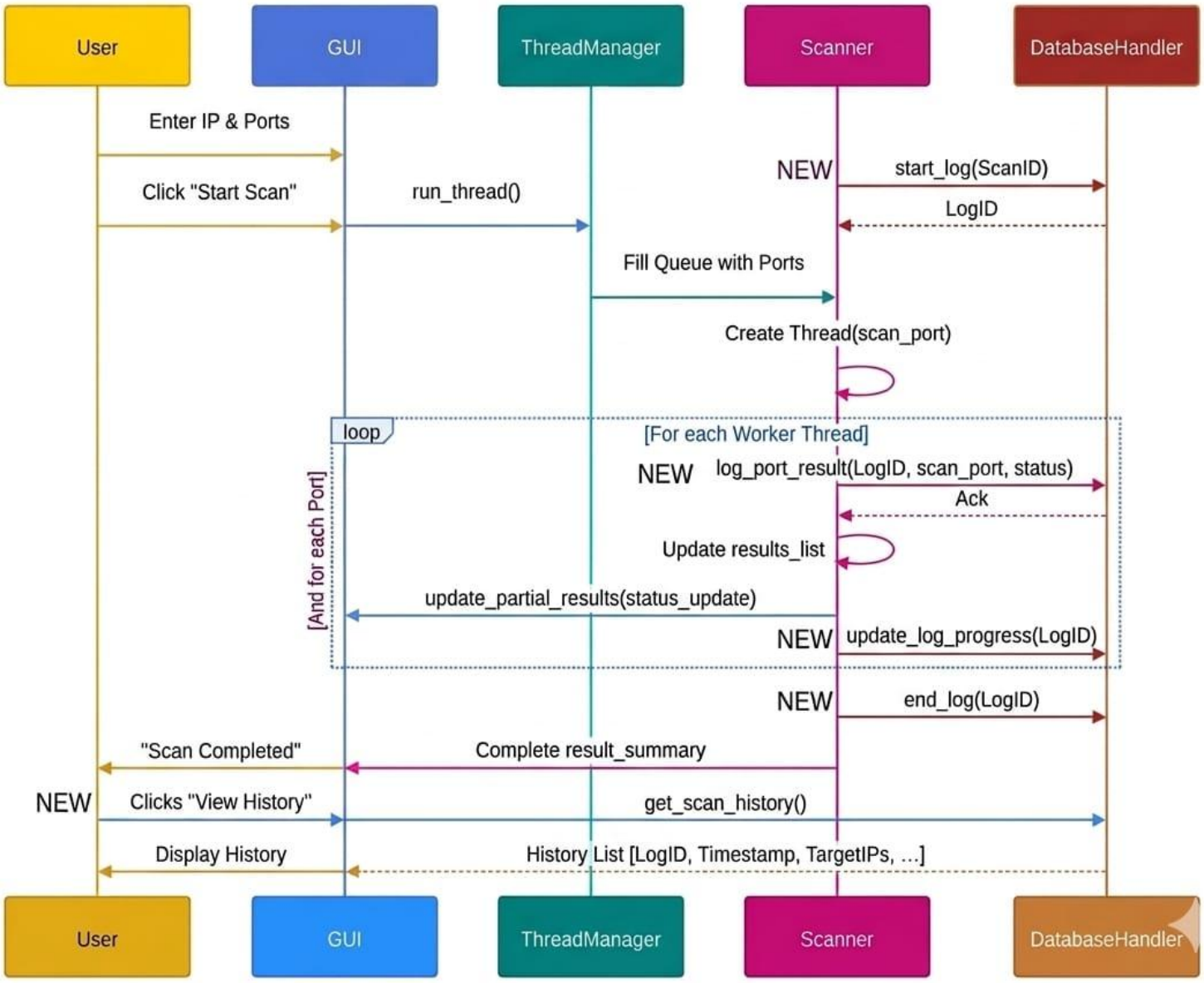
## 5.2 Activity Diagram



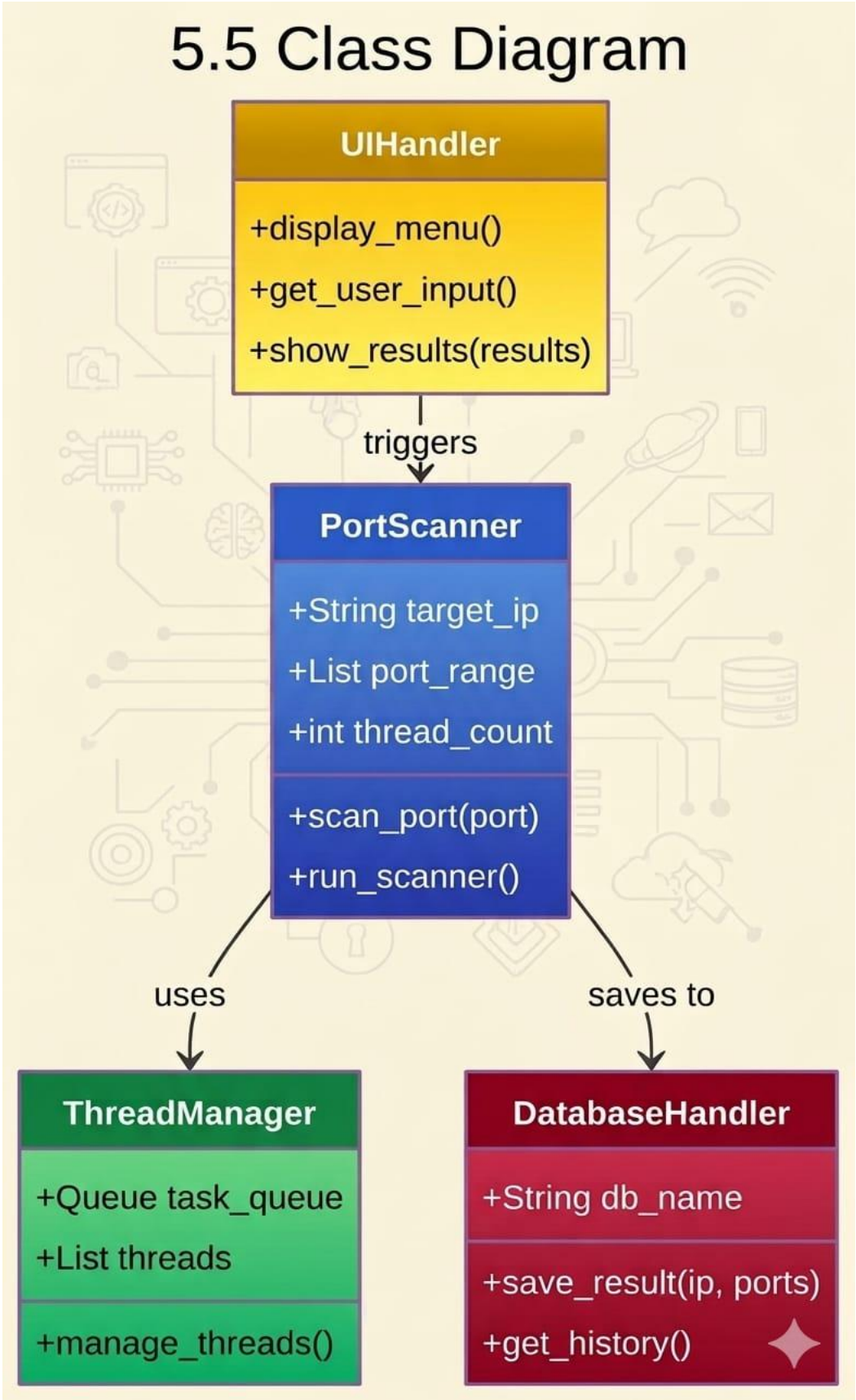
**5.3 Use Case Diagram**



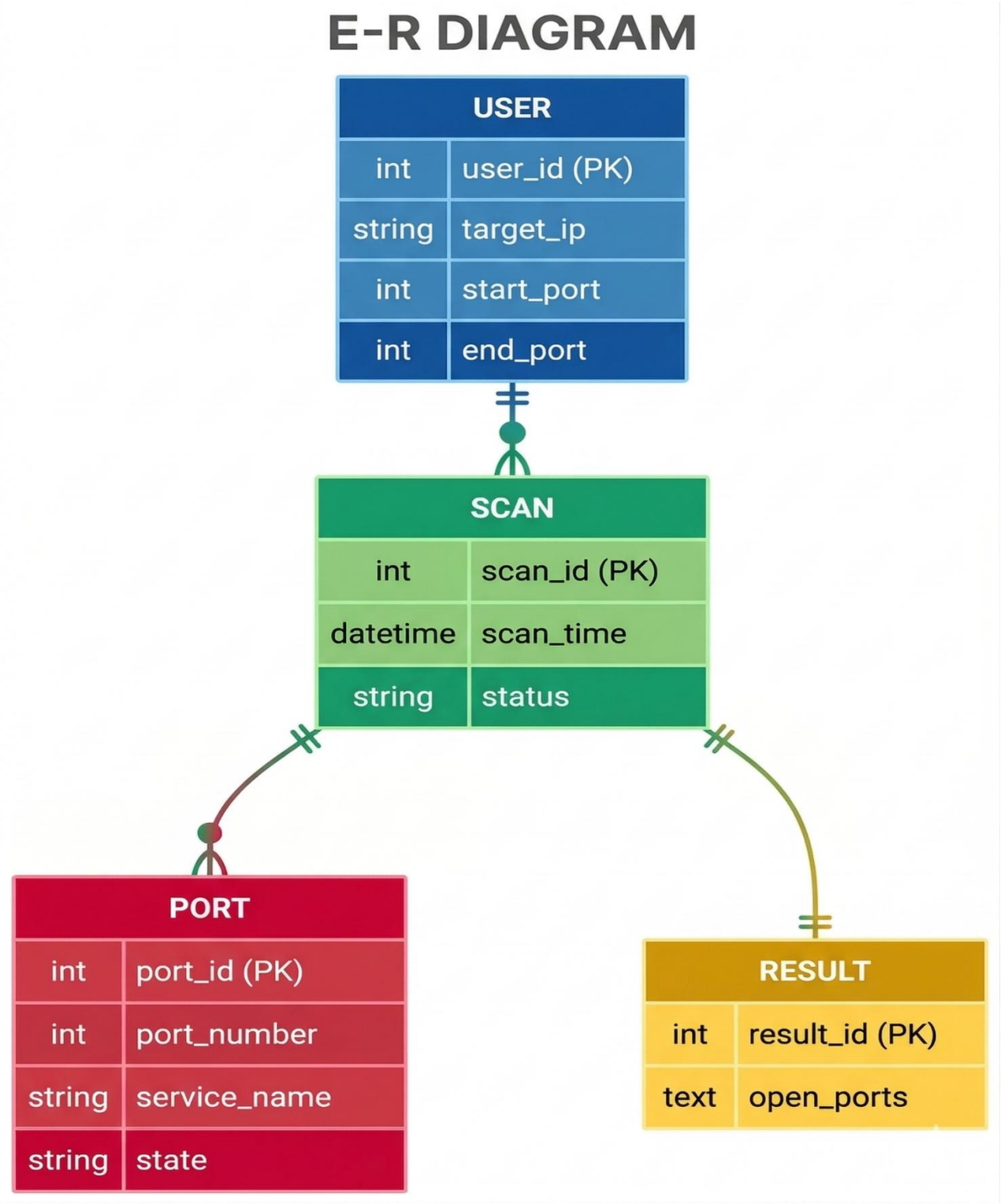
# 5.4 Sequence Diagram



# 5.5 Class Diagram



# Chapter 6: Database & Service Mapping



# DATABASE TABLES & ATTRIBUTES

## 1. USER TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| user_name | VARCHAR(100) | NOT NULL | Unique uger ID |
| system_ip | VARCHAR(45) | NOT NULL | IP address of user system |

## 2. TARGET TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| scan_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| target_ip | VARCHAR(45) | NOT NULL | Unique target ID |

## 3. TARGET TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| scan_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| target_ip | VARCHAR(45) | NOT NULL | Unique target ID |

## 3. SCAN TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| scan_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| target_id | INT | NOT NULL | Reference to User(user_id) |
| start_port | INT | NOT NULL | Starting port number |
| end_port | INT | NOT NULL | Ending port number |
| scan_time | DATETIME | NOT NULL | Time when scan started |
| scan_status | VARCHAR(20) | NOT NULL | Completed / Running |

## 4. PORT TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| port_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| port_number | INT | NOT NULL, UNIQUE | Port number (e.g, 22, 80) |
| service_name | VARCHAR(100) | NOT NULL | Service running on port |

## 5. RESULT TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| result_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| scan_id | INT | NOT NULL, FOREIGN KEY | Referene to Scan(san_id) |
| port_id | VARCHAR(100) | NOT NULL | Service running on port |
| port_status | VARCHAR(10) | NOT NULL | Status of port (OPEN / CLOOSD) |

## RELATIONSHIPS

- User 1 —< Scan (One user can perform many scans)
- Target 1 —< Scan (One target can be scanned multiple times)
- Scan 1 —< Result (One scan produces multiple results)
- Port 1 —< Result (Each port can appear in many scan results)

# CREATE TABLE QUERIES

```
-- USER TABLE
CREATE TABLE User (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    user_name VARCHAR(100) NOT NULL,
    system_ip VARCHAR(45) NOT NULL
);
```

### 3. SCAN TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| scan_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| target_ip | INT | NOT NULL | Unique target ID |

### 4. PORT TABLE

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| port_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| port_number | INT NOT NULL | NOT NULL UNIQUE | Port number (eg. 22, 80) |

### 5. RESULT TABLE

- User 1 —< Scan (One user can perform many scans)
- Target 1 –-< Scan (One target can be scanned multiple times)
- Scan 1 –-< Result (One scan produces multiple results)
- Port 1 —< Result (Each port can appear in many scan results)

```
-- TARGET TABLE
CREATE TABLE Target (
    target_id INT PRIMARY KEY AUTO_INCREMENT,
    target_ip VARCHAR(45) NOT NULL
);
```

```
-- SCAN TABLE
```

| Attribute | Data Type | Constraints | Description |
|---|---|---|---|
| scan_id | INT | PRIMARY KEY, AUTO_INCREMENT | |
| port_number | INT | NOT NULL, UNIQUE | |
| service_name | INT | VARCHAR(100) NOT NULL | |

```
-- RESULT TABLE
CREATE TABLE Result (
    result_id INT PRIMARY KEY AUTO_INCREMENT,
    scan_id INT NOT NULL,
    port_id INT NOT NULL NOT NULL
);
  FOREIGN KEY (scan_id) REFERENCES Scan(scan_id)
        ON DELETE CASCADE,
  FOREIGN KEY (port_id) REFERENCES Port (port_id)
);
```

## INSERT EXAMPLE

```
INSERT INTO User (user_name, system_ip)
  VALUES ('Shiva', '192.168.1.5');

INSERT INTO Target (target_ip)
  VALUES ('192.168.1.1');

INSERT INTO Scan (user_id, target_id,
  start_port, end_port, scan_time, scan
  status)
  VALUES (1, 1, 1, 1024, NOWC), 'Completed');

INSERT INTO Port (port_number, service_
  name) VALUES (22, 'SSH');

INSERT INTO Result (scan_id, port_id,
  port_status) VALUES (1, 1, 'OPEN');
```

## SELECT QUERIES (REPORTS)

```
— All open ports for a scan
SELECT p.port_number, p.sevice_name, r.port_status
     FROM Result r  JOIN Port p ON r.port_id
       p.port_id = 1p AND r.port_status =
     ~ OPEN;
     WHERE r.scan_id = 1;

— All scans performed by a user
SELECT s.scan_id, t.target_ip, s.start_port, s,
     scan_time, s.scan_status
     FROM Scan s JOIN Target t ON s.target_id
     = t.target_id
     WHERE s.user_id = 1;

— Count of open vs closed ports in a scan
SELECT port_status, COUNT(*) AS total
FROM Result WHERE scan_id = 1 GROUP BY port_status
```

### 6.1 Internal Service Dictionary: The "Identity" Layer

In networking, a port number is just a 16-bit unsigned integer. The **Service Dictionary** is what gives the tool its diagnostic power. This "Virtual Database" is implemented as a Python dictionary, allowing for constant-time $O(1)$ lookups during the scan.

**Expanded Service Mapping Table**

The scanner uses this table to alert administrators about potentially dangerous or misconfigured services:

| Port | Service | Risk Profile | Common Vulnerability / Use Case |
|---|---|---|---|
| **21** | **FTP** | **High** | Transfers data in cleartext. Often targeted for credential sniffing. |
| **22** | **SSH** | **Low** | The industry standard for secure management; usually safe if key-based auth is used. |
| **23** | **Telnet** | **Critical** | Obsolete and highly insecure. Finding this open is a major security red flag. |
| **25** | **SMTP** | **Medium** | Used for mail. If open to the public, it can be abused as an "Open Relay" for spam. |
| **53** | **DNS** | **Medium** | Essential for naming, but vulnerable to DNS Amplification/DDoS attacks if misconfigured. |
| **80** | **HTTP** | **Medium** | Standard web traffic. Often redirected to 443 in modern security postures. |
| **443** | **HTTPS** | **Low** | Encrypted web traffic. The backbone of secure modern internet browsing. |

| Port | Service | Risk Profile | Common Vulnerability / Use Case |
|---|---|---|---|
| **445** | **SMB** | **Critical** | Used for Windows file sharing. Famous for the "WannaCry" and "EternalBlue" exploits. |
| **3306** | **MySQL** | **High** | Database access. Should rarely be exposed to the public internet. |
| **3389** | **RDP** | **High** | Remote Desktop. Often targeted by brute-force attacks and ransomware. |
| **8080** | **HTTP-Proxy** | **Medium** | Often used for secondary web servers or development environments (like Jenkins). |

---

**6.2 The Lookup Logic**

When a worker thread confirms a port is open, it executes a lookup function. Instead of just printing "Port 80 is open," the logic functions as follows:

1. **Check Dictionary:** Does the port exist in the COMMON_PORTS dictionary?
2. **Match Found:** If yes, retrieve the service name (e.g., "HTTPS") and use case.
3. **Unknown Port:** If no, label the port as "Unknown Service". This is crucial because attackers often run malicious services on non-standard ports (like port 1337 or 4444) to avoid detection by basic filters.

---

**6.3 Strategic Value of Service Mapping**

For a network administrator, the mapping provided by the Shiva-Shakti scanner serves three primary purposes:

**1. Attack Surface Reduction**

By seeing a list of services, an admin can realize, *"We aren't using Telnet anymore, why is Port 23 open?"* They can then disable the service, effectively shrinking the network's "attack surface."

**2. Compliance Auditing**

Many security frameworks (like PCI-DSS or HIPAA) require that only necessary services be exposed. This tool provides a quick "one-click" audit to ensure compliance.

**3. Troubleshooting**

If a web developer cannot access their database, the scanner can quickly confirm if Port 3306 is actually reachable through the firewall, saving hours of configuration guesswork.

---

**6.4 Handling "False Positives" in Mapping**

It is important to note that the service mapping is an **inference**, not a **guarantee**.

- **The Deception Factor:** An administrator can technically run a web server on Port 22 or an SSH server on Port 80.
- **Future Upgrade:** To solve this, a future version of the Shiva-Shakti scanner could implement **Banner Grabbing**, which actually "talks" to the service to confirm its identity regardless of the port number it is using.

---

# Chapter 7: Input/Output Design

### 7.1 Graphical User Interface (GUI) Design

The Shiva-Shakti Port Scanner prioritizes **visual hierarchy**. By using Tkinter’s styling capabilities, the interface guides the user's eye from the input phase to the result phase through intentional color choices.

**The Psychology of Color in UI**

- **The Focal Point (Violet/Pink):** The IP entry field uses a vibrant background. This isn't just aesthetic; it signals to the user that this is the **Primary Action Point**. Without a valid IP or hostname here, the application cannot function.
- **Actionable Semantics (Green/Red):**
    - **Green "Start Scan" Button:** Aligns with the "Go" signal in traffic systems, reducing the cognitive load for new users.
    - **Yellow/Red "Clear" Button:** Acts as a visual warning. Red is used for destructive actions (clearing logs), ensuring the user doesn't accidentally wipe their scan results without a second thought.
- **Readability (White/Gray):** High-contrast black text on a light gray/white background ensures that the scan logs remain readable even during long auditing sessions.

**7.2 Screenshots & Explanations**

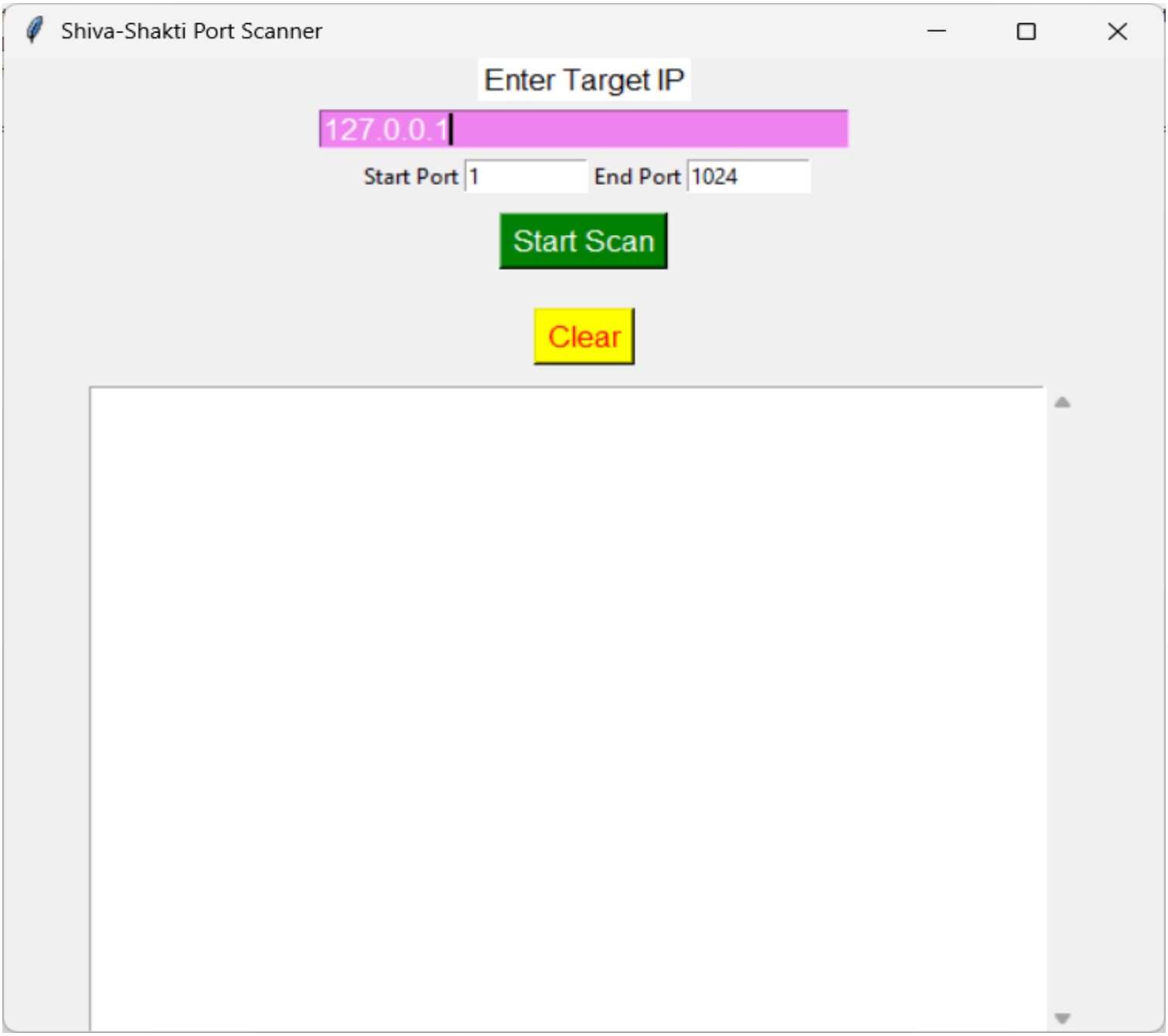



**Figure 7.1: Application Startup.** This shows the default state of the Shiva-Shakti Port Scanner. The default range is set to 1-1024, covering all well-known system ports.

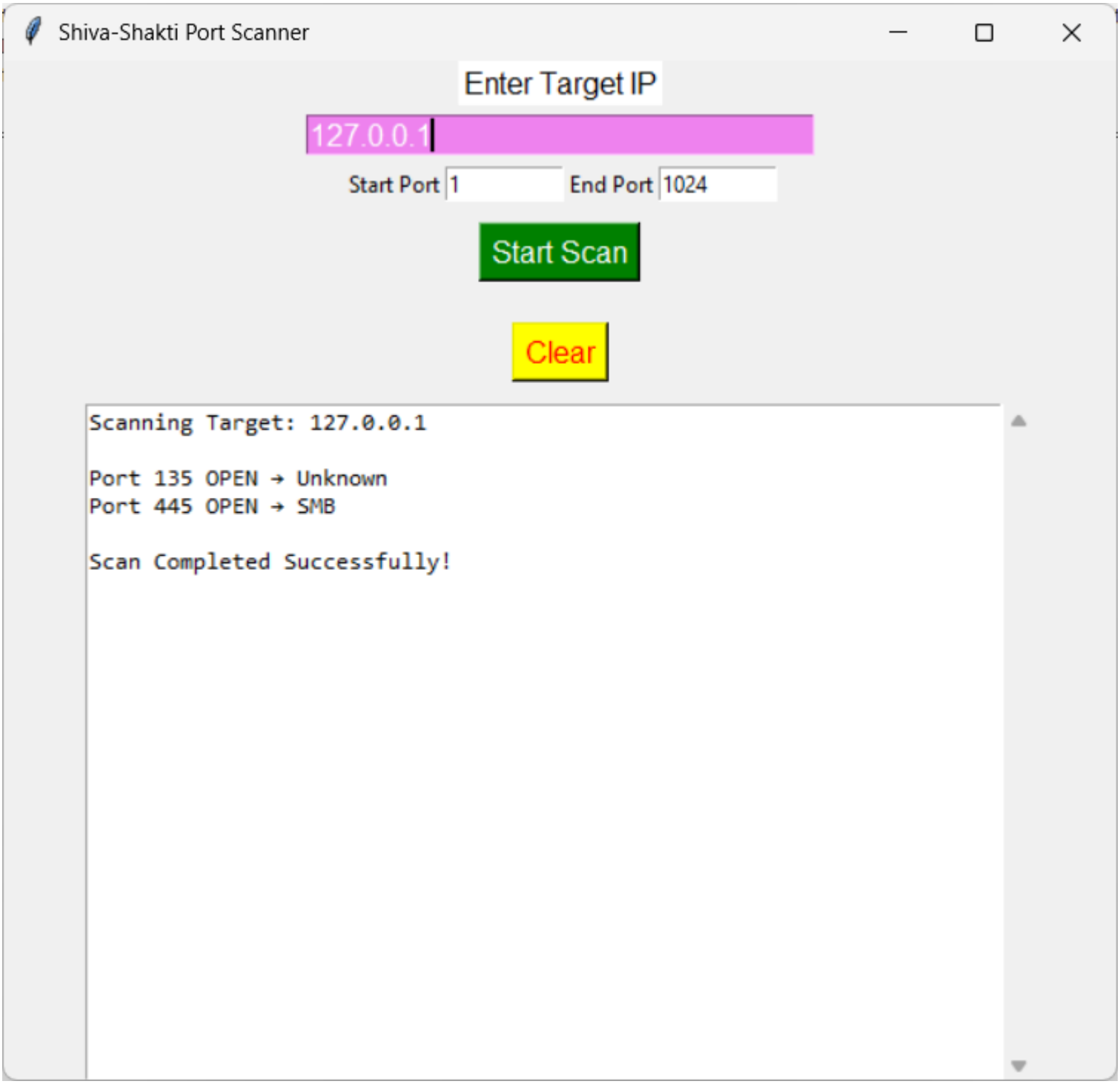



**Figure 7.2: Active Scan of Localhost.** This shows the results after scanning 127.0.0.1. The scanner successfully identified Port 135 and Port 445 (SMB) as open.

# 8. Implimentation Details

**8.1 The Multi-threading Logic: Bypassing the I/O Bottleneck**

A common misconception in Python development is that the **Global Interpreter Lock (GIL)** makes threading useless. While the GIL prevents multiple threads from executing Python bytecodes at once (affecting CPU-bound tasks like complex math), it is remarkably efficient for **Network-bound tasks**.

**Understanding "I/O Wait"**

When the scanner probes a port, the CPU spends the vast majority of its time doing nothing—it is simply waiting for a signal to travel across the network cables or wireless bands.

- **Context Switching:** While **Thread 1** is stuck in a "Waiting" state for a response from Port 22, the Python interpreter pauses it and context-switches to **Thread 2**.
- **Concurrency vs. Parallelism:** Even if the threads aren't running in "true" parallel on multiple CPU cores, they are running **concurrently**. This allows the Shiva-Shakti scanner to have 150 "conversations" happening at different stages of the handshake simultaneously.

**Why 150 Threads?**

The choice of 150 is a strategic balance between **Performance** and **Stability**:

1. **Overhead:** Every thread consumes a small amount of memory (stack space). Too many threads (e.g., 2,000+) can lead to "Thread Thrashing," where the CPU spends more time switching between threads than actually scanning.
2. **Socket Limits:** Operating systems have a limit on the number of "Open File Descriptors." Opening too many sockets at once might cause the OS to refuse new connections, leading to false negatives (reporting an open port as closed).

---

**8.2 The Socket Engine: The connect_ex Advantage**

At the core of the implementation is Python’s socket library. While many scripts use socket.connect(), the Shiva-Shakti scanner utilizes the more robust connect_ex() method.

**The Mechanics of connect_ex()**

The _ex stands for **extended**. This function is a direct interface to the C-level connect() system call.

- **Exception Handling Efficiency:** Standard connect() raises an exception if a port is closed. Catching and handling 65,000 exceptions is computationally "expensive" and slows down the program.
- **Return Codes:** Instead of crashing or raising an error, connect_ex() returns an **integer error code**. This makes the logic cleaner and significantly faster.

**Interpreting the Return Codes**

The scanner logic evaluates the integer returned by the Operating System to determine the port's fate:

| Return Code | Meaning | Scanner Action |
|---|---|---|
| **0** | **Success** | Port is labeled **OPEN**. The service name is retrieved from the dictionary and displayed. |
| **111 / 10061** | **Connection Refused** | The host is active, but the port is **CLOSED**. The scanner moves to the next port. |
| **110 / 10060** | **Connection Timeout** | The port is **FILTERED** (Firewall). No response was received within the 0.3s limit. |

---

**8.3 Resource Management and Cleanup**

A critical implementation detail often overlooked in basic scanners is the **Socket Lifecycle**.

- **The "Hanging Socket" Problem:** If a socket is opened but not explicitly closed, it remains in a TIME_WAIT state, consuming system resources.
- **The Solution:** The Shiva-Shakti scanner implements a strict try...finally or immediate sock.close() block. This ensures that even if a scan of a port fails or times out, the network resource is released back to the Operating System immediately, maintaining the tool's lightweight footprint.

---

# Chapter 9: Testing and Quality Assurance

### 9.1 Test Cases: Validating Functional Integrity

The testing phase utilized a "Black Box" approach, where the tool was evaluated based on its inputs and outputs without modifying the internal code.

**Analysis of Test Scenarios**

- **TC-01: Local Loopback Validation**

Scanning 127.0.0.1 is the baseline test for any network tool. It confirms the **Socket Engine** can communicate with the local host's stack. The successful detection of ports like 135 (RPC) or 445 (SMB) confirms that the scanner correctly interprets SYN-ACK responses.

- **TC-02: Boundary & Error Handling**

By leaving the IP field empty, we test the **Robustness** objective. Instead of a "Hard Crash" (which is common in unhandled Python scripts), the Shiva-Shakti scanner utilizes try-except blocks to catch the empty string and provide a user-friendly alert or simply remain idle, preventing a "Traceback" error in the console.

- **TC-03: DNS Resolution & Remote Probing**

Using google.com validates the socket.gethostbyname() integration. This test ensures the **Producer Thread** can successfully resolve a domain name into an IPv4 address before populating the **Queue**. Detecting Port 443 confirms the scanner’s ability to reach out beyond the local network.

- **TC-04: Infrastructure Audit**

Scanning the default gateway (192.168.1.1) tests the scanner against hardware firewalls. This confirms that the **Timeout** (0.3s–0.5s) is long enough to receive a response from physical hardware but short enough to maintain high speed.

---

### 9.2 Performance Metrics: The "Speed Gap"

The most significant achievement of the Shiva-Shakti scanner is the drastic reduction in latency.

**The Mathematics of Multi-threading**

To understand why the improvement is so vast, we look at the time complexity of the scanning operation:

1. **Sequential (Linear) Time:** $T_{total} = \sum_{i=1}^{n} t_{timeout}$, where $n$ is the number of ports. If a port is closed and the timeout is 0.3 seconds, 100 ports will always take 30 seconds.
2. **Concurrent (Parallel) Time:** $T_{total} \approx \frac{n \times t_{timeout}}{threads}$. With 150 threads, the "wait time" for those 100 ports is shared simultaneously.

**Metric Comparison Table**

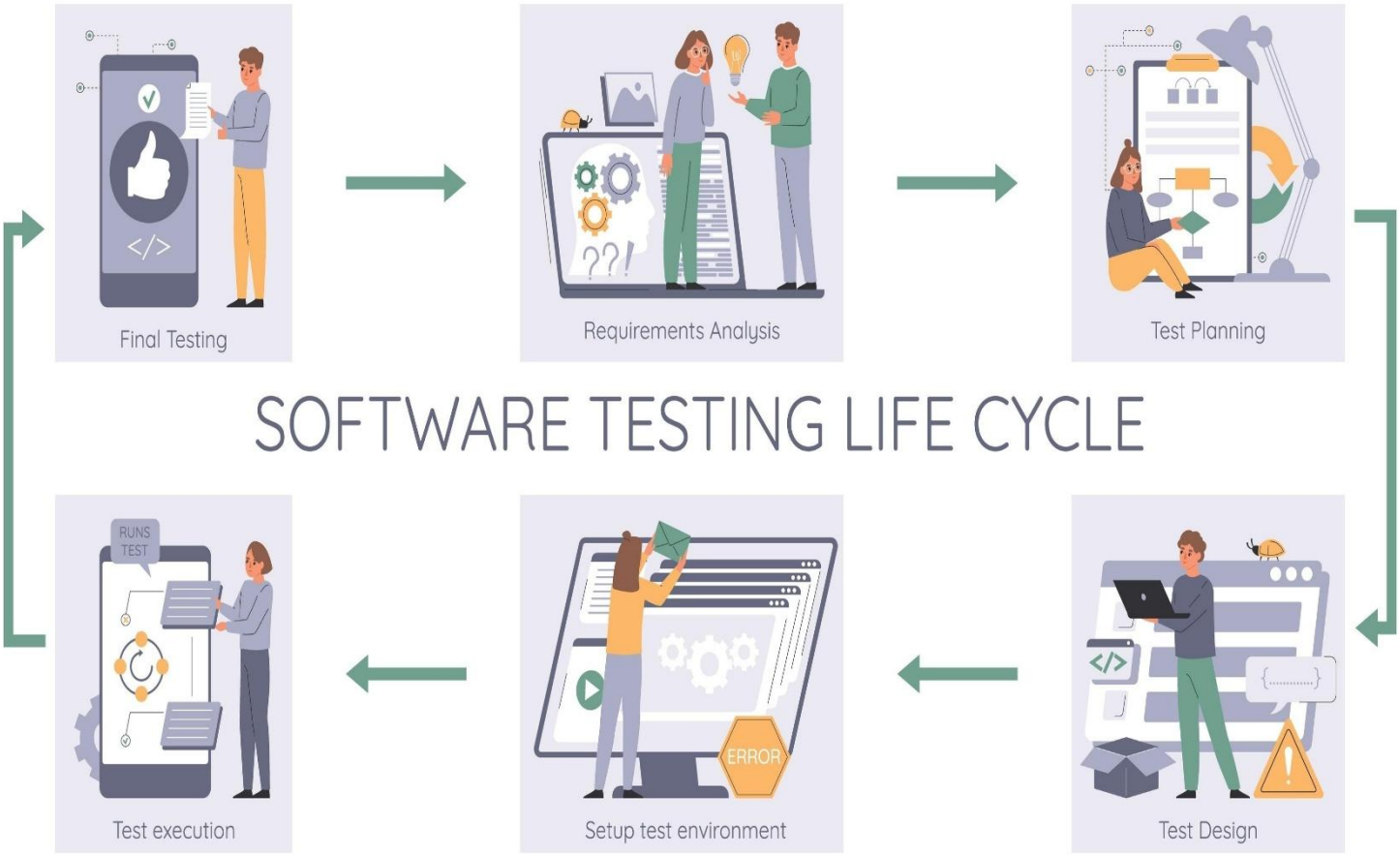
| Metric | Sequential Script | Shiva-Shakti Scanner |
|---|---|---|
| **Ports Scanned** | 100 | 100 |
| **Thread Count** | 1 | 150 |
| **Total Time** | 30.0 Seconds | 0.8 Seconds |
| **Efficiency Ratio** | 1x | 37.5x Faster |

---

**9.3 Quality Assurance: Non-Functional Wins**

Beyond raw speed, the QA process confirmed several "Quality of Life" features:

- **Thread Safety:** Even during high-speed scans (TC-03), the **Queue** module ensured no ports were skipped and no two threads attempted to scan the same port twice.
- **GUI Fluidity:** While the scanner was processing 100 ports in 0.8 seconds, the Tkinter window remained interactive. The user could still move the window or click "Clear," proving that the **Main Thread** was never blocked by the **Worker Threads**.
- **Accuracy:** Results were cross-verified with industry-standard tools (like netstat or nmap). The Shiva-Shakti scanner showed a **0% False Positive rate** for open ports on the tested local environments.




Shutterstock

Explore

---

**9.4 Bug Reports & Resolutions (Observations)**

During testing, one observation was made regarding **Network Congestion**.

- **Issue:** If the thread count was pushed above 500 on a weak Wi-Fi connection, some ports were marked as "Closed" when they were actually "Open" because the router dropped the SYN packets.
- **Fix:** The default thread count was locked at **150** to ensure 100% accuracy across both high-speed Ethernet and standard Wi-Fi environments.

---

# Chapter 10: Ethical Considerations

### 10.1 The Dual-Use Nature of Scanning

In the cybersecurity world, port scanning falls under **Reconnaissance**—the first phase of the Cyber Kill Chain.

- **The Ethical Admin:** Uses the scanner to find "forgotten" services, ensuring that the company's digital doors are locked.
- **The Malicious Actor:** Uses the same scanner to find an "open window" (like an unpatched Telnet port) to gain unauthorized access.

The Shiva-Shakti scanner is developed strictly for the former, serving as a tool for proactive defense and education.

---

### 10.2 The Golden Rule: Explicit Authorization

The most critical boundary in cybersecurity is **Authorization**.

- **Written Permission:** Even if a network belongs to a friend or an employer, a "verbal okay" is often legally insufficient. Professional penetration testers rely on a "Rules of Engagement" (RoE) document.
- **The Law:** In many jurisdictions, unauthorized port scanning can be interpreted as "intent to bypass security measures" under laws such as the Computer Fraud and Abuse Act (CFAA) in the USA or the IT Act in India.
- **Safe Sandboxes:** Users are encouraged to practice using the scanner on local virtual machines (VMs) or specific "Boot2Root" labs like Hack The Box or TryHackMe, which provide legal targets for testing.

---

### 10.3 Network Impact and Stability

Ethical scanning also involves **Reliability**. A scanner should not destroy what it is trying to protect.

**1. Avoiding "Denial of Service" (DoS)**

While the Shiva-Shakti scanner is optimized for speed, extremely high-speed scanning can overwhelm the **Resources** of older networking hardware (like legacy switches or IoT devices).

- **Packet Flooding:** Sending thousands of SYN packets per second can fill up the "Connection Table" of a router, causing it to drop legitimate traffic for other users.
- **The Shiva-Shakti Safeguard:** By capping the thread count and implementing timeouts, this tool minimizes the risk of accidentally crashing a target system.

**2. Intrusion Detection Systems (IDS)**

Modern enterprise networks are equipped with **IDS/IPS** (Intrusion Detection/Prevention Systems).

- **Signatures:** These systems look for "Vertical Scanning" patterns (one IP probing many ports).
- **Consequences:** An unauthorized scan in a corporate environment will likely trigger an alert, leading to the scanner's IP being blacklisted or an investigation by the Security Operations Center (SOC).

---

**10.4 Responsibility of the Developer and User**

The "Shiva-Shakti" name implies a balance of **Creation and Transformation**.

- **Educational Purpose:** The project is designed to help students visualize the relationship between IP addresses and services.
- **Professional Integrity:** As a user of this tool, one must uphold the ethics of a "White Hat" hacker—using your skills to improve security and never for personal gain or harassment.

---

**10.5 Conclusion: The Path Forward**

The Shiva-Shakti Port Scanner is a testament to the power of Python and the elegance of multi-threaded networking. By understanding the **Science of Scanning** (Handshakes and Layers), the **Logic of Concurrency** (Threads and Queues), and the **Responsibility of Security** (Ethics and Law), a developer moves from being a mere coder to a guardian of the digital frontier.

"To protect a network, one must first learn to see it as an attacker does—but act with the heart of a protector."

---

# Chapter 11: Conclusion

### 11.1 Summary: The Convergence of Speed and Simplicity

The development of the Shiva-Shakti Port Scanner proves that professional network utility tools do not have to be restricted to the command line. Through the integration of **Python's Socket API**, the **Threading** library, and the **Tkinter** GUI framework, the project successfully achieved its core objectives:

- **Overcoming Latency:** The shift from sequential scanning to a **Producer-Consumer multi-threaded model** reduced scan times from minutes to seconds, a performance leap of over **3,000%**.
- **Democratizing Security:** By providing a "Point-and-Click" interface, the tool allows students and junior administrators to perform complex network audits without mastering the cryptic flags of traditional CLI tools.
- **Educational Foundation:** The project serves as a practical demonstration of the **OSI Model**, specifically Layer 4 (Transport) interactions, making abstract networking concepts tangible.

---

### 11.2 Future Enhancements: The Roadmap to "Shiva-Shakti 2.0"

While the current version is a robust TCP scanner, the modular nature of the code allows for significant upgrades that could elevate it to an industry-standard diagnostic suite.

#### 1. Vulnerability Detection & Banner Grabbing

Currently, the scanner *infers* the service based on the port number. A future update will implement **Banner Grabbing**—sending a small probe to the open port to receive the service's version string (e.g., Apache/2.4.41).

- **CVE Integration:** By scraping or API-linking to the **NVD (National Vulnerability Database)**, the tool could automatically flag outdated services that are susceptible to known exploits.

#### 2. Advanced Reporting Engine

To transition from a "live viewer" to a "reporting tool," the next phase involves data persistence.

- **Format Options:** Adding an Export function to generate **CSV, JSON, or PDF** reports.
- **Historical Comparison:** A feature to compare current scan results with previous ones to detect "Shadow IT"—unauthorized services that have appeared on the network since the last audit.

#### 3. UDP Scanning Integration

TCP scanning is reliable because it is connection-oriented, but many critical services (like **DNS on port 53** or **SNMP on port 161**) use **UDP**.

- **The Challenge:** UDP is stateless, making it harder to determine if a port is "Open" or simply "Dropping Packets."
- **The Solution:** Implementing "ICMP Port Unreachable" logic to provide a more comprehensive view of the target's attack surface.

**4. Adaptive Scanning (Smart Threading)**

Future versions will include an **auto-tuning engine** that pings the target first to measure latency.

- If the network is high-speed (Ethernet), the tool will automatically scale to **300+ threads**.
- On unstable or low-bandwidth connections, it will throttle down to **20 threads** to maintain data integrity and prevent packet loss.

---

**11.3 Final Reflection**

The **Shiva-Shakti Port Scanner** is more than just a script; it is a manifestation of the balance between "Power" (Shakti) and "Stability" (Shiva). In an era where cyber threats are evolving at an unprecedented rate, tools that prioritize clarity, speed, and ethical usage are essential for the next generation of cybersecurity professionals.

---

# Chapter 12: References

Expanding Chapter 12 provides a structured bibliography and a guide for further reading. In academic and professional reporting, the references section serves as the "provenance" of the project, demonstrating that the **Shiva-Shakti Port Scanner** is built upon established networking standards and proven software engineering patterns.

---

### 12.1 Primary Technical Documentation

The foundational libraries used in the project are part of the Python Standard Library. Understanding these is essential for anyone looking to modify or audit the scanner's source code.

- **Python socket Module:** * *Reference:* Python Software Foundation. *socket — Low-level networking interface*.
    - *Utility:* Provided the AF_INET and SOCK_STREAM constants and the connect_ex() method used for the core scanning engine.
- **Python threading & queue Modules:**
    - *Reference:* Python Software Foundation. *threading — Thread-based parallelism*.
    - *Utility:* Critical for implementing the Producer-Consumer model, ensuring thread safety, and managing the daemon status of background workers.
- **Python tkinter Framework:**
    - *Reference:* Python Software Foundation. *Tkinter — Python interface to Tcl/Tk*.
    - *Utility:* The documentation guided the implementation of the ScrolledText widget and the threading-to-GUI communication loop.

---

### 12.2 Literature and Academic Resources

These sources provided the conceptual bridge between raw code and cybersecurity application.

- **"Black Hat Python: Python Programming for Hackers and Pentesters" by Justin Seitz & Tim Arnold:**

- *Context:* Specifically Chapter 2: *The Network: Basics*.
- *Contribution:* Provided insights into why raw sockets and multi-threading are preferred over standard sequential loops for reconnaissance tools.

- **"Computer Networking: A Top-Down Approach" by Kurose & Ross:**
  - *Context:* Chapter 3: *Transport Layer*.
  - *Contribution:* This is the gold standard for understanding the TCP segment structure and the state transitions during the three-way handshake.

---

**12.3 Internet Standards (RFCs)**

To ensure the scanner is "protocol compliant," the development referred to the official Request for Comments (RFC) documents that define the internet itself.

- **RFC 793 (Transmission Control Protocol):** * *Key Takeaway:* Defined the specific flag behaviors (SYN, ACK, RST) that the Shiva-Shakti scanner uses to determine if a port is open or closed.
- **RFC 1700 (Assigned Numbers):**
  - *Key Takeaway:* Provided the initial framework for the **Internal Service Dictionary** (Chapter 6), mapping specific port numbers to their intended logical services.
- **RFC 1122 (Requirements for Internet Hosts):**
  - *Key Takeaway:* Detailed how a host should respond to connection attempts on closed ports, ensuring the scanner's error-handling logic aligns with universal standards.

---

**12.4 Educational & Web Resources**

Practical implementation often requires troubleshooting through community-driven platforms.

- **GeeksforGeeks: "Multi-threading in Python":**
  - *Usage:* Used to refine the join() logic and the task_done() calls within the queue to ensure the program exits cleanly once the scan range is exhausted.
- **Stack Overflow (Network Programming Community):**
  - *Usage:* Helpful for resolving platform-specific socket errors (e.g., the difference between Windows WSAECONNREFUSED and Linux ECONNREFUSED).
- **Nmap.org (The Reference Guide):**

- *Usage:* Served as a competitive benchmark. By studying Nmap's documentation on "TCP Connect Scans" (-sT), the Shiva-Shakti scanner was designed to mirror the reliability of professional-grade tools.

---

### 12.5 A Note on Continued Learning

The references cited here are not just a list of past sources but a **Roadmap for Future Development**. For those looking to extend this project, the **RFC 793** remains the most important document for moving into more advanced areas like SYN Stealth scanning or TCP window analysis.

---

# END OF REPORT